

GENETICKÉ PROGRAMOVÁNÍ

→ evoluce programů

1. náhodná populace programů
- 2. fitness = výsledky běhů programů na test datech
3. nová populace programů
→ selekce podle fitness, křížení, mutace

Reprezentace programů

1. Lineární

- byte code, posloupnost instrukcí
 - jednoduché operátory (lineární vektory)
 - rychle
 - mutace a křížení generuje mnoho nevalidních programů
- artificial life, evo. of bots, control code in games

2. Grafová

- DAG → orientovaný acyklický graf
vrcholy = proměnné, hrany = operace
- vhodné pro obvody, automaty, neuronky, reinforcement learning
- komplikované operátory, hlavně křížení

3. Stromová

- reprezentace programu syntaktickým stromem
terminály = proměnné, konstanty
neterminály = operace
- selekce: většinou turnajová
- mutace: podstrom ← náhodný podstrom
- křížení: prohození podstromů (typicky častěji v neterminálu)

- inicializace :

- a) grow → generují random strom než dosáhnou
max # vrcholů

- b) full → generují strom z neterminálů do max hloubky,
pak random terminály

- c) ramped half-and-half → půl na půl grow a full

- mutace :

- lepší využít víc typů :

- a) náhodná nebo systematická mutace konstant

- aritmetická
 - iterace hill-climbingu nebo jiná optimalizace
 - na 1 nebo i všech nn. konstant stromu

- b) výměna neterminálů stejné arity

- c) permutace

- d) nahrazení neterminálu terminálem

- e) zmenšování stromu (práce s podstromy)

- křížení

- a) prohození 2 podstromů

- b) uniformní crossover

- velikost podstromů klesá s délkou běhu

- křížení čím dál víc lokální

- C = podobná oblast v obou podstromech

- $\forall$ vrchol z C má konstantní pr. na crossover point

- vrcholy uvnitř C prohozeny bez podstromů

- vrcholy na okraji C prohozeny i s podstromy

- C → checkování uzlů stejné arity

- prohození buď v c.p. nebo 2 random

- c) context preserving crossover

- prohození části cesty od kořene do listu

- vrcholy se stejnými „coordinates“

- seeding = inicializace s částečným řešením
 - hrozí předčasná konvergence
- bloating → roste velikost stromů
 - limit pro hloubku nebo # vrcholů
 - ↳ např. penalizaci ve fitness nebo kontrolami
 - anti-bloat operátory

ADF = automatically defined functions

- rozdělení na podprogramy
- typicky main a ADF podstromy
 - (ADF call = heterminál v main tree)
- zulašit evoluce na main a ADFs

Developmental GP

- evoluce stromů, které popisují program na vytvoření grafu
- elektrické obvody, neuronky, atd.
- operace pro strukturu (př. paralelní, sériové, 3-tier)
 - operace pro prvky (př. rezistor, kondenzátor, ...)

Gramatická evoluce (GE)

- bezkontextová gramatika
 - pravidla s terminály a neterminály, výchozí symbol
- $\forall$ odvození lze reprezentovat derivačním stromem
- genotyp = lineární posloupnost - pravidla gramatiky
 - ↳ pořadí pro aplikaci
- fenotyp = syntax tree (derivační strom po redukci)

Kartézský GP

- evoluce grafů reprezentovaných popisem souřadnic (2D)
- fenotyp = DAG
- genotyp = lineární posloupnost uzlů
 - vrchol, vstup, výstup, funkce
- point mutation, aritmetické křížení
- selekce $(1+L)$, L většinou 4

EVOLUCE KOOPERACE

- darwinismus je kompetitivní
- kooperace snižuje fitness jedince
- evoluce na úrovni skupin
 - co jedinci, kteří podvádějí a nepomáhají?
- kin selection
 - zachování identických genů v blízkých příbuzných
 - co ale altruismus cizinců / jiných druhů?
- evoluce na úrovni jednotlivých genů

vlastnosti úspěšných strategií:

- 1) niceness → don't deceive first
 - 2) provocability → quickly punish deception
 - 3) forgiveness → quickly calm down
 - 4) clarity → be simple so others can understand you
- favor coop in payoffs + big chance of iterated PD → kooperace

TUNING, META-EVOLVE

- parametry evoluce $\rightarrow$ vel. populace, velikost turnaje, prob. operatorů

a) tuning

- trial & error, grid search
- experience and good luck

b) controlling

- parametry nebývají nezávislé
- změna v čase nebo v závislosti např. na varianci fitness

Strategie:

1) fixed

- změna v závislosti na čase

< deterministická $\rightarrow$ např. snižování pr. mutace o 0,1% + generaci
stochastická $\rightarrow$ simulované žíhání

2) adaptivní

- podle kvality řešení, statistické vl. populace, ...

3) self-adaptive

- evoluce evoluce (evoluce parametrů)

Příklady:

- 1) reprezentace $\rightarrow$ podle diversity populace měním přesnost čísel

- 2) mutace

- Gaussovská mutace $x(t+1) = x(t) + N(0, \sigma)$ $\swarrow$ variance
- determinismus $s(t) = 1 - 0,9 \left(\frac{t}{T} \right)$
- evoluční strategie $\rightarrow$ optimal success rate, evoluce parametrů

- 3) selekce

- Boltzmann tournament

- prob. horšího závisí na poměru rozdílů fitness a teploty
- teplota se časem snižuje, zvyší se u lok. optima

4) populace

- velikost populace závisí na fitness jedince
 - θ jedinec má danou délku života podle fitness

NEUROEVOLUCE

= metoda pro nastavování parametrů neuronových sítí pomocí evoluce (váhy, topologie, ...)

trénování vah:

- přímé $\rightarrow$ váhy ve vektoru, floating point GA, ev. str., stand. op.
 - pomalejší než gradient based loc. s. alg. jako back propagation
 - lze paralelizovat
 - lze použít pro reinforcement tasks

hledání architektury:

- fitness = performance estimation
 - $\rightarrow$ několikrát natreénování vah
- direct encoding $\rightarrow$ binární matice kód. propojení
- grammatical encoding $\rightarrow$ reprezentace gramatikou
- cellular encoding $\rightarrow$ kódování jako program popisující stavbu NN
 - $\rightarrow$ přidání neuronu, rozdělení, prohození synapsí...

GNARL

- reprezentace sítě jako grafu $\rightarrow$ GP
- fixní input a output nodes
- evolují hidden nodes a spoje
- evoluce parametrů a struktury současně
 - $\leftarrow$ mutate struktury - $\dagger$ node bez spojení, $\dagger$ spojení s váhou 0, odstranění
 - mutate vah - biased

→ síla mutace se mění s teplotou (simulované žití)

• selection = rodice jsou lepší polovina populace

NEAT

= neuroevolution of augmenting topologies

• reprezentace NN jako seznam hran

- vrcholy

- váhy

- edge id

- disabled flag

• křížení jen na hranách se stejným id (nerozbije síť)

• similarity measure na vektorech edge ids

- niching = podobné sítě jsou stejného druhu

→ fitness počítám relativně v rámci druhu

→ chrání nové topologie než se vytvoří váhy

→ structural changes rozbíjí fitness ale jsou potřeba

HyperNEAT

• evoluce vah pro fixní NN

• substrate = váhy jsou generované fci

- fci: souřadnice x, y 2 neuronů → váha spojení mezi nimi

$S: \mathbb{R}^4 \rightarrow \mathbb{R}$ tzv. hyperkrychle do $\mathbb{R}$ (proto HyperNEAT)

- reprezentované CPPN evolovane NEATem

• CPPN = compositional pattern-producing networks

→ typ NN s diverse activ. fci v $\forall$ uzlu

→ NN-like struktura s uzly v DAGu

→ $\forall$ node má jinou fci ze sloupíků

- sigmoidy, polynomiály, sin, cos, tan, gaussian...

↳ bohatý universe possible substrate functions

Neuroevolve pro hluboké sítě

NAS

= neural architecture search

Search space:

- lineární
 - vrstvy NN
 - max # vrstev, typ vrstev
 - hyperparametry: # jednotek pro fully connected layers
velikost kernelu, # filtrů, strides, pooling
- DAG
 - cells (bloky) ∇ normální (zachovají dimensionalitu)
redukovat
 - kompletní architektura → stack bloků
 - reduced search space
 - transfer cells for other tasks

Strategie:

- EVA, random search, Bayesovská optimalizace, reinforcement learning, gradient-based methods

fitness: single / multi-objective

ENAS

= efficient NAS

- search space = 1 DAG
- search = výběr podgrafu DAGu
- sdílení vah → netreningů & model od začátku

Deep NEAT

- jednotka není neuron ale vrstva DNN
 - opravy, aby matchovaly velikosti vstupu a výstupu

CoDeep NEAT

- koevoluční Deep NEAT
- 2 populace:
 - ◀ modules - bloky, malé DNNs
 - blueprints - grafy užívající moduley
- lze použít pro LSTM (Long short-term memory recurrent NN)
 - 2 special mutace:
 - ◀ změna vnitřní struktury jednotky
 - hledání uspořádání jednotky ve vrstvě

Hill climbing NAS pro CNN

- začnu s malou předtreňovanou sítí
- HC loop → aplikace mutací / morfujících operací
 - prohloubení sítě (+ convoluční blok)
 - rozšíření sítě (vyšší # of channels)
 - přidání skip connection between layers
- limitovaný čas trénování child net

MEMETICKÉ ALGORITMY

meme = myšlenka, idea, která přžívá ve společnosti

· evoluční algoritmus s vylepšovací procedurou

→ ∇ jedinec vylepšen lokálním prohledáváním

→ HC, simulované žitání, gradient search

→ memetické koncepty pro meta-heuristiky

co s výsledkem?

1) Lamarckismus

→ když pomocí LS najdu lepšího jedince, uložím ho do populace

→ změníme genotyp podle fenotypu

2) Baldwinismus

→ změním fitness na tu lepší, ale neměníme genotyp

↳ „potenciál“ původního jedince

DYNAMIC FITNESS

· změna fitness fce v průběhu

→ může záviset na podmínkách nebo genotypu

· klasické GA špatné → ztrácím diverzitu

modifikace:

1) **diploidní reprezentace**

→ ∇ jedinec má 2 sady chromozomů

2) **hypermutace**

→ zvýšení prob. mutace, když klesne fitness v populaci

3) **random migrace**

→ při poklesu fitness přidám random generated jedince

4) udržování diversity

- snížení selection pressure
- sub-populace → křížení v rámci nich („ostrůvky“)
- crowding, niching, dynamic species

BIOLOGICKY PŘESNÉ EA

Druhy (species)

- problém s poklesem diversity
- non-random mating

niching → křížení pouze podobných jedinců
→ fitness relativní v rámci podobných

crowding → turnaje podobných jedinců

• měření podobnosti jedinců

→ binární repr. hamming

• hranice niche → threshold na podobnost, dynamické

• co-evolve → kontext fitness

↳ kooperace

predator - prey

PROHLÉDAVACÍ ALG.

Local search

→ blízko lepší řešení pomocí lokální změny

Tabu search

- tabu list = seznam nedávno navštívených stavů
- kombinace s local search (HC)
- short-term memory → uložení tabu listu, nescyklím se
- long-term memory → směřuji do neprozkoumaných oblastí
- tabu list nejsou celá řešení ale kroky k nim (např. výměna hrany)
- tabu tenure = jak dlouho zůstane řešení v tabu listu
→ statické / dynamické
- aspiration criteria = kdy můžu použít krok z tabu listu
(když nejsou jiné možné kroky)
- frequency memory = globální counter pro t řešení
→ kolikrát bylo navštíveno

Scatter search

- snaha vylepšit globální search tím, že se soustředí na diverzitu
→ reference set = diverzifikovaní jedinci s vysokou fitness
1. populace $P(0)$ a r.s. $R(0)$
 2. křížení $R(t-1) \rightarrow P(t)$
 3. mutace $P(t)$
 4. výběr $R(t)$

Novelty search

- diversity jako objective → hledání pouze novelty
- sparseness = součet vzdáleností od k nearest neighbors
- behavior space = space of unique behaviors (fenotypy)

- udržují archiv of novel indiv.

→ přidám jedince s vysokou sparseness

ACO

= ant colony optimisation

- hledání cest v grafu

- každá hrana má atraktivitu a množství feromonů

- daemon action = local search on path (1 mraveneček)

AS - ant system

→ řeší TSP

→ feromony evaporují s nějakou rate

ACS - ant colony system

→ globální update feromonů pomocí nejlepšího mravence

min-max → kladný feromoni udržování v limitech

CMA-ES

Black box optimization

- minimalizují fci, kterou neznám (nemůžu určit gradient)

→ jediná info = evaluated search nodes

- search cost = # vyhodnocení fce

- metody řešení: EVA, ES, CMA-ES, local search
stochastic search

stochastic search:

udržují prob. distr. $P_{\theta}(x)$

→ pro efficient sampling hodnot fee

1. nasamplují vstupy z $P_{\theta}(x)$

2. vyhodnotí f na vzorcích

→ data $\{(x_i, f(x_i))\}_{i=1, \dots, n}$

3. aktualizují θ podle dat (θ = parametr)

→ opakuji do konvergence θ

θ = propaguje se mezi iteracemi → obsahuje historii, definuje budoucnost

GA: θ = populace rodičů

ES: θ = Gaussian with mean & variance

S.Ž: θ = current point a teplota

CMA-ES

= covariance matrix adaptation

$\theta = (x, \sigma, C, p_{\sigma}, p_C)$

EVOLUČNÍ PROGRAMOVÁNÍ

- genotyp = fenotyp
- agent jako konečný automat
 - ↳ predikuje další stav prostředí, podle toho vrátí akci
- vstup = stav prostředí (znak)
- fitness = úspěšnost predikce (absolute/mean square error)
+ velikost automatu
- nové křížení
- mutace:
 - změna symbolu stavu
 - změna hran mezi stavy
 - přidání / odebrání stavu

- $1/2$ populace nahrazena novými automaty

SGA

zjednodušení SGA:

1. random populace, jedinec = binární řetězec délky l
2. dokud nemá novou populaci:
3. 2 jedince (selektce)
4. křížení s p_c
5. jednoho nahradím !!! zjednodušení
6. mutace t bitů s p_m
7. nový jedinec do populace

- t jedinec číslo mezi 0 a 2^l
- populace v čase t : 2 vektory délky 2^l : $p(t)$ a $s(t)$

pop. comp.

↓

sel. prob.

↓

$p_i(t)$ = v populaci je jedinec hodnoty i (rel. frequency)

$s_i(t)$ = pravděpodobnost výběru jedince i

- matice F : $F(i,i) = f(i)$, $F(i,j) = 0$
- pak $s(t) = \frac{F \cdot p(t)}{\sum_i F(i,i) p(t)}$

- operator G , který dělá iteraci SGA $\rightarrow p(t+1) = G \cdot p(t)$
 $\rightarrow s(t+1) = G \cdot s(t)$

$$G = F \circ M \rightarrow \text{mutace a křížení (mixing)}$$